

Zurich ServiceNow AI Platform Capabilities

Last updated: January 16, 2026

Some examples and graphics depicted herein are provided for illustration only. No real association or connection to ServiceNow products or services is intended or should be inferred.

This PDF was created from content on docs.servicenow.com. The web site is updated frequently. For the most current ServiceNow product documentation, go to docs.servicenow.com.

Company Headquarters

2225 Lawson Lane
Santa Clara, CA 95054
United States
(408)501-8550

CMDB classifications and class dependency

CMDB classifications are groups of CIs that share attributes and are stored in their own class table. Classifications let administrators to define a class hierarchy for the CIs within the CMDB.

A CI class refers to the actual table name in the instance database. In that context, 'CI type' is a friendly name that a CI is known by, such as computer, router, or printer.

One of the characteristics of a CMDB class is its dependency on other classes. A class can be independent or dependent, which determines the dependency or independence of the class CIs.

Independent CIs

CIs from an independent class, such as Linux Server CIs, have their own identity, exist by themselves, and aren't dependent on any other CIs.

Dependent CIs

CIs from a depended class have relationships with CIs from other classes and exist meaningfully only if these relationships exist. Dependent CIs don't exist on their own in the absence of the dependent relationship. For example:

- Network Adapter CIs don't exist meaningfully without the Hardware CIs that contain them.
- Application CIs don't exist on their own without the Server CI they are hosted on.

Identification processes in the [Identification and Reconciliation engine \(IRE\)](#) CMDB feature extensively use CIs dependency classification. To find out the dependency classification of a class, open the Identification Rule page for a class in [CI Class Manager](#). To establish CI dependency, use the CI Class Manager to specify [dependent relationship rules](#) for a CI class.

For more information, see [Identification rules](#).

- [Dependent CIs management](#)

A life cycle update for a CI affects its dependent CIs. For example, when the CI that a dependent CI depends on is deleted, the dependent CI becomes orphan with no further use. To maintain the integrity and health of the CMDB, the system applies cascade-cleanup processes to dependent CIs that are affected by a life cycle update.

- [CMDB record types](#)

The CMDB contains the following major record types.

- [Related Lists of CI components](#)

Related lists in CI records display additional components contained by that CI, such as disk drives on a server and the rules that control the behavior of a network router.

- [Create a CI class](#)

Create a CI class (table) that is an extension of an existing CI class. Then create identification and reconciliation rules for the new class.

- [Reclassify a CI](#)

You can upgrade, downgrade, or switch the class of a CI by modifying its **Class** attribute.

- [Delete CIs](#)

You can use the CI Class Manager to delete CIs that are no longer needed.

- [View and edit class definitions and metadata](#)

Use the CI Class Manager as a central location to explore the CMDB class hierarchy, CI table definitions, and class CIs. View the details of each table such as its label and fields, relationships, and all related metadata definitions.

- [Update class list in the Principal Class filter](#)

A Principal Class is a designation for those CMDB classes that are most important or critical for foundational data health and governance in the organization. Manage the list of classes in the Principal Class filter

so that those classes are prioritized for tracking, health, certification, lifecycle management, and class list views.

Dependent CIs management

A life cycle update for a CI affects its dependent CIs. For example, when the CI that a dependent CI depends on is deleted, the dependent CI becomes orphan with no further use. To maintain the integrity and health of the CMDB, the system applies cascade-cleanup processes to dependent CIs that are affected by a life cycle update.

For information about independent and dependent CIs, see [CMDB classifications and class dependency](#).

To ensure that dependent CIs are properly managed after deleting or archiving CIs, you must:

1. Enable dependent CIs management as described on this page.
2. Manually approve the [CMDB Data Manager](#) tasks that dependent CIs management generates, or configure those tasks to not require a review or an approval.

Note: An Orphan dependent CI in the context of Data Manager is different from an orphan CI in the context of CMDB Health. An orphan dependent CI within the context of Data Manager belongs to a dependent class, and is missing the dependent relationship. The definition of an orphan CI in the context of CMDB Health is broader, and includes any CI that matches CMDB Health orphan rules. For more information about orphan CIs in CMDB Health, see [CMDB Health KPIs and metrics](#).

Examples of dependent CIs needing cascade-cleanup:

- Tomcat application (T1) runs on a Linux Server (L1) and contains a WAR file (W1). When L1 is deleted, T1 and W1 become orphan dependent CIs.
- Network Adapters become orphan dependent CIs when the hardware itself is deleted.
- A Linux Server has retired and is set with end of life, the assumption is that if the server is no longer operational then applications which depend on it shouldn't be operational either.

Enable dependent CIs management

To enable dependent CIs management:

- Configure your environment for CMDB Data Manager. For details about how to configure the environment for CMDB Data Manager and the CMDB Data Manager own prerequisites, see [Working with CMDB Data Manager](#).
- Ensure that the `cmdb.dependent.ci.cascade.op.enabled` system property is set to **true** (true by default). This property enables cascade operations and doesn't exist in the base system. Therefore, to view or modify the property value, you must first [add it to the System Properties \[sys_properties\]](#) table.
- To enable cascade-retirement processes, ensure that the system property `cmdb.dependent.ci.cascade.retire.enabled`, which is used to manage cascade-retirement of CIs, is set to true (false by default). The business rule After BR: Track "Retired" CIs, that checks for downstream CI dependencies, runs only if this property is set to true. This property doesn't exist in the base system and to view or modify it, you must first [add it to the System Properties \[sys_properties\]](#) table.

Cascade-cleanup operations apply only from when you enable the dependent CIs management feature. To apply cascade-cleanup to orphan dependent CIs that already existed in the CMDB before enabling the feature, see [Cascade-cleanup existing orphan dependent CIs](#).

Use of CMDB Data Manager to perform cascade-cleanup operations

Dependent CIs management processes use the [CMDB Data Manager](#) to process the life cycle updates for the dependent CIs in the CMDB Dependent CI Ledger [`cmdb_dependent_ci_ledger`] table in the following ways:

1. Requesting and getting approvals from users for the life cycle updates for the CIs.
2. Performing the actual delete, archive, or retire CI updates after these operations are approved.

The system generates tasks for the 'Dependent CI - Deletion', 'Dependent CI - Archive', and 'Dependent CI - Retire' Data Manager policies, with the CIs that are ready to be deleted, archived, or retired as target CIs.

These policies are set with the respective 'On Demand' policy types. Then, a Data Manager admin or user, according to the Managed By Group setting, must review and approve these tasks before the Data Manager applies the respective subflow to the dependent CIs.

To automatically approve those tasks, use the CMDB Data Manager to clear the **Needs Review** flag of the respective Data Manager policies. Those tasks will then run without requiring any user intervention.

Cascade-retire dependent CIs

When a CI is set to retire, dependent CIs management data processes attempt to cascade-update all the CIs depending on that CI, also to retire.

1. CI is updated to retire as the CI's Life Cycle Stage attribute is set to **End of Life** according to CSDM standards.
2. Business rule After BR: Track "Retired" CIs runs (only if the system property `cmdb.dependent.ci.cascade.retire.enabled` is **true**) to check if the CI belongs to a class which has any downstream dependencies.

If yes: The CI is added to the CMDB CI End Of Life Ledger [`cmdb_ci_end_of_life_ledger`] table.

3. For each CI in the CMDB CI End Of Life Ledger [`cmdb_ci_end_of_life_ledger`] table, the scheduled job CMDB Cascade Retire Dependent CIs checks the CI's relationships in the CI Relationship [`cmdb_rel_ci`] table. If there is a dependent relationship with a dependent CI, then that dependent CI is added to the CMDB Dependent CI Ledger [`cmdb_dependent_ci_ledger`] table with the **Retire** action.

For more information about dependent relationship rules, see [CMDB dependent relationship rules](#).

4. Scheduled Job CMDB DependentCI Policy Processor runs (if the system property `cmdb.dependent.ci.cascade.retire.enabled` is **true**) to process the CIs in the CMDB Dependent CI Ledger [`cmdb_dependent_ci_ledger`] table. Those CIs are set as being ready to retire upon approval using the [CMDB Data Manager](#). A task is created in the CMDB Data Manager against the Dependent CI Retire policy to actually retire the CIs.

Cascade-archive dependent CIs

Archiving a CI can leave its dependent CIs as orphans in the CMDB. To prevent the accumulation of stale data, the system applies cleanup processes that cascade-archive those orphan dependent CIs.

Orphan dependent CIs are not immediately archived. When a CI is archived, all the CI relationship records for that CI in the [cmdb_rel_ci] table, are also archived. The system then checks for any dependent CIs that were orphaned as a result of this archiving. Any CIs in the dependency chain that are identified as orphan dependent CI, are checked for any of the following conditions. CIs that meet any of these conditions are not orphan dependent CIs and therefore will not be archived:

- The CI is an unhandled duplicate CI (the CI is associated with an unresolved de-duplication task).
- The CI has multiple parent CIs.
- The CI belongs to an excluded class. Excluded classes are stored in the CMDB Dependent CI Class Exclusion [cmdb_dependent_ci_class_exclusion] table. In the base system, that table is pre-populated with some classes such as cmdb_ci_vm, cmdb_ci_vmware_instance, and other VMware-related classes. You can manage the set of classes that are exempt from management of orphan dependent CIs, by adding or removing records to that table.

Eventually, only those CIs that these conditions do not apply to are added to the CMDB Dependent CI Ledger [cmdb_dependent_ci_ledger] table. Those CIs are set as being ready for archival using the [CMDB Data Manager](#), upon approval.

Cascade-delete dependent CIs

Deleting a CI can leave its dependent CIs as orphans in the CMDB. To prevent the accumulation of stale data, the system applies cleanup processes that cascade-delete those orphan dependent CIs.

1. CI is updated to be deleted as the CI's Life Cycle Stage attribute is set to End of Life according to CSDM standards.
2. All relationships in the cmdb_rel_ci table, associated with the deleted CI, are deleted.

3. Business rule After BR: CMDB Dependent CI Delete runs (only if the system property `cmdb.dependent.ci.cascade.op.enabled` is **true**) to check each of the deleted relationships in the CI Relationship [`cmdb_rel_ci`] table, whether the other CI (parent or child) in the relationship is an orphan dependent CI.

If yes: Those orphan dependent CIs are added to the CMDB Dependent CI Ledger [`cmdb_dependent_ci_ledger`] table and set with Action = Delete.

4. Scheduled Job CMDB DependentCI Policy Processor runs (if the system property `cmdb.dependent.ci.cascade.retire.enabled` is true) to process the CIs in the CMDB Dependent CI Ledger [`cmdb_dependent_ci_ledger`] table. Those CIs are set as being ready to be deleted upon approval using the CMDB Data Manager. A task is created in the CMDB Data Manager against the Dependent CI Delete policy to actually delete the CIs.

Orphan dependent CIs aren't immediately deleted. Prior to deleting each of the CIs relationship in the CI Relationship [`cmdb_rel_ci`] table, the system checks if other CI in the relationship belongs to a dependent class. Any CIs in the dependency chain that are identified as dependent on the deleted CI, are checked for any of the following conditions. CIs that meet any of these conditions aren't orphan dependent CIs and therefore aren't marked for deletion:

- The CI is an unhandled duplicate CI (the CI is associated with an unresolved de-duplication task).
- The CI has multiple parent CIs.
- The CI belongs to an excluded class. Excluded classes are stored in the CMDB Dependent CI Class Exclusion [`cmdb_dependent_ci_class_exclusion`] table. In the base system, that table is pre-populated with some classes such as `cmdb_ci_vm`, `cmdb_ci_vmware_instance`, and other VMware-related classes. You can manage the set of classes that are exempt from management of orphan dependent CIs, by adding or removing records to that table.

Eventually, only those CIs that these conditions don't apply to are added to the CMDB Dependent CI Ledger [`cmdb_dependent_ci_ledger`] table. Those CIs are set as being ready for deletion using the [CMDB Data Manager](#), upon approval.

Extraneous relationships

For the delete and archive operations, the system also tracks CIs' extraneous relationships in the Dependent CI Relations Evaluation Config [cmdb_dependent_ci_extra_rels_config] table. Extraneous relationships are those relationships that aren't in the CI's chain of dependency and deleting or archiving them depends on the value of the cmdb.dependent.ci.extra.rel.check system property. This property is set to **true** by default, in which case extraneous relationships aren't deleted or archived. You can set the property to **false** to delete and archive extraneous relationships.

The cmdb.dependent.ci.extra.rel.check property doesn't exist in the base system and to view or modify the property value, you must first [add it to the System Properties \[sys_properties\]](#) table.

Cascade-cleanup existing orphan dependent CIs

When enabling the dependent CIs management feature, the cascade-cleanup operations apply only from when the feature is enabled. However, it might be necessary to apply a similar cascade-cleanup operation to orphan dependent CIs that already existed in the CMDB before the feature was enabled.

[Activate](#) the Cleanup Orphan CIs scheduled job to perform a one-time cascade-cleanup of orphan dependent CIs across the CMDB. The Cleanup Orphan CIs scheduled job checks throughout the entire CMDB to identify any orphan dependent CIs. These CIs are then processed in the same manner that CIs that are cascade-deleted are processed. CIs that are ready to be deleted are added to the CMDB Dependent CI Ledger [cmdb_dependent_ci_ledger] table. The CMDB Data Manager is then leveraged as described in the [Use of CMDB Data Manager to perform cascade-cleanup operations](#) section.

The Cleanup Orphan CIs scheduled job is intended to run only once, after which the job deactivates itself. Depending on the size of the CMDB, it might take the Cleanup Orphan CIs scheduled job several days to complete.

Sample scenario: cascade-retire

1. Enable dependent CIs management.

2. In CMDB Data Manager, create and publish a Retire policy. For example, retire an independent Linux Server CI with relationships to the dependent CIs Network Adapter and File System.
3. Wait for the scheduled job CMDB Data Manager Retire Policy Processor to run, or run it manually.
4. In the Data Manager overview page, select the Open task card and approve the retirement task (Approval=Approved). Change the state of the task to 'Work in progress' if it doesn't change automatically. This will retire the Linux Server CI and add it to the cmdb_ci_end_of_life_ledger table.
5. Run the scheduled job CMDB Cascade Retire Dependent CIs.

This scheduled job processes each CI in the cmdb_ci_end_of_life_ledger table, and queries the relationships in the cmdb_rel_ci table for this CI. If a relationship exists with other dependent CI, then those dependent CI are added to the cmdb_dependent_ci_ledger table with the settings of state = 'Ready' and action = 'Retire'.
6. Wait for the scheduled job CMDB DependentCI Policy processor to run, or run it manually. This scheduled job creates a task against the OOB Dependent CI Retire policy, targeting the CIs in the cmdb_dependent_ci_ledger table.
7. In the Data Manager overview page, select the Open task card and approve the task described with "Dependent CI – Retire..".

Related tasks

- [Create a CI class](#)
- [Reclassify a CI](#)
- [Delete CIs](#)
- [View and edit class definitions and metadata](#)
- [Update class list in the Principal Class filter](#)

Related reference

- [CMDB record types](#)
- [Related Lists of CI components](#)

CMDB record types

The CMDB contains the following major record types.

CMDB record types

Record types	Description
Configuration Item (CI)	Any computer, device, or service in the CMDB. A CI's record includes all of the relevant data, such as manufacturer, vendor, location, etc. Configuration items can be created or maintained either using tables, lists, and forms within the platform, or using the Discovery application.
Relation Type	A defined relationship between a CI and either another CI, a user, or a group. Relation types are defined twice, once from the perspective of the child CI and once from the parent CI's perspective. For example, a parent CI that powers a child CI uses relation type Powers::Is Powered By. Example relation types include In Rack::Rack contains, Log Reviewed by::Reviews logs for, or Backup done by::Does backups for. CMDB relationships can be established using Discovery or

Record types	Description
	using the tables, lists, and forms within the platform. The CMDB form has a specific Related Items toolbar optimized for modifying relationships.

Related tasks

- [Create a CI class](#)
- [Reclassify a CI](#)
- [Delete CIs](#)
- [View and edit class definitions and metadata](#)
- [Update class list in the Principal Class filter](#)

Related concepts

- [Dependent CIs management](#)

Related reference

- [Related Lists of CI components](#)

Related Lists of CI components

Related lists in CI records display additional components contained by that CI, such as disk drives on a server and the rules that control the behavior of a network router.

When Discovery runs, the Related List is populated with the components that Discovery finds running on the CI. The CI record might show different lists from scan to scan, depending on whether or not Discovery found the component.

By default, the Related Lists only display those components that are associated with that CI in the CMDB that has been discovered by the

last scan. Components that are recorded in the CMDB but are not discovered in a scan, are deemed absent and do not appear in the list.

There are two types of components that appear in the Related List: components that are CIs themselves (such as hard disks), and components that are not (serial numbers and rules). The default filter condition in the breadcrumbs for components that are CIs is **[Status] [!=] [Absent]**. The filter condition for components that are not CIs is **[Absent] [=] [false]**.

For example, a router can have several Related Lists affected by these filter conditions, including routing rules, disk drives, interfaces, and network adapters. Only those components found during the last Discovery appear in these Related Lists.

Icons in related lists

Any of the following icons may appear in lists of CI related items.

	For currently active incidents against this configuration item
	For currently active problems against the configuration item
	For currently active changes against the configuration item that are not covered in the past, current, pending changes. For example, a request to update the operating system on a server that is currently in progress may display this icon.
	For changes that were recently completed against the configuration item. changes with an "Actual_end_date" in the past.
	For changes that are planned soon against the configuration

	item. changes with an "Actual start date" in the future.
	For currently active changes against the configuration item that have an "Actual start date"
	For outages that were recently completed against the configuration item. outages with an "end" date in the past.
	For outages that are planned soon against the configuration item. outages with a "begin" date in the future.
	For currently active outages against the configuration item that have a "begin" date in the past and no "end" date
	This will only show up in the Tree view and indicates that a configuration item that is downstream has at least one of the above issues against it.

The system looks five calendar days into the past and seven calendar days into the future when looking at recent outages and changes.

- [Teams related list](#)

The Teams related list associates a user group to a CI based on group type, providing flexibility in tracking the different types of groups assigned to a CI. The Teams related list appears on CI forms for CIs of the Service [cmdb_ci_service] class and its descendent classes such as the Service Instance [cmdb_ci_service_auto] class.

Related tasks

- [Create a CI class](#)

- Reclassify a CI
- Delete CIs
- View and edit class definitions and metadata
- Update class list in the Principal Class filter

Related concepts

- [Dependent CIs management](#)

Related reference

- [CMDB record types](#)

Teams related list

The Teams related list associates a user group to a CI based on group type, providing flexibility in tracking the different types of groups assigned to a CI. The Teams related list appears on CI forms for CIs of the Service [cmdb_ci_service] class and its descendent classes such as the Service Instance [cmdb_ci_service_auto] class.

Using the Teams related list can be useful if many data sources are used in the organization, and when using the [IntegrationHub ETL](#).

In the base system, the Teams related list contains group types that match the fields:

- Approval Group
- Change Group
- Managed by Group
- Support Group

When you set a group assignment on a CI form of one of those classes, that group assignment is automatically synchronized with the Teams related list. If you set an assignment group for an application service, a relationship record is created to represent the new group assignment for

the CI. The Teams related list on such CI forms, always shows the current settings for the various group assignments for the CI.

If there are multiple groups assigned to a CI, then on the **Teams** related list on a CI form, you can designate one of those groups as primary. When an incident involving that CI is created, the incident is assigned to that primary group. Only one group type can be designated as primary for a CI at any given moment.

As an example, use the Teams related list for an application service with the following characteristics:

- Requires access to a database
- Runs on Linux servers
- Has a group assigned to manage the software portion of the actual application

In that example, you can track all of these group assignments by adding the appropriate group types and setting CIs with the new custom group types.

By-directional synchronization

When you set or modify the value of one of the assignment group fields on a Service CI form, then the Teams related list is updated with that change, adding a Teams related record if a corresponding one doesn't exist. For example, when you set an empty **Support group** field to **Database San Diego** and save the form, then the system adds a Teams record in which Group type is **Support Group** and **User group** is **Database San Diego**.

In the opposite direction, any change in a Teams related record, where Group type corresponds one of the group assignment fields in the CI, is synchronized to the corresponding field. For example, if you set User group to **CAB Approval** in the Support Group group type, the **Support group** field on the CI form is updated with the **CAB Approval** value.

Deleting a Teams record doesn't affect the group assignment fields in the CI.

Add the Teams related list to a Service CI form

By default, the tab for the Teams related list doesn't appear in the Related Lists section on Service CI forms. You can add the Teams related list by selecting the form's Additional actions menu, selecting **Configure** and then **Related Lists**. In the Available list that appears, move **Teams** to the Selected list and then select **Save**.

Add a custom group type to the Teams related list

The Teams related list lets you add custom group types that are needed in your organization, extending the initial list of supported group types.

Before you begin

Role required: itil_admin, asset, or cmdb_admin

About this task

To add a group type, you must modify the dictionary definition of the group_type column in the cmdb_rel_team table.

Procedure

1. On a Service CI form, select the **Teams** related list.
2. In the **Columns** tab, select the **Group type** column.
3. On the Dictionary Entry form, in the **Choices** related list tab, insert a new row for a new Teams choice such as **Datacenter Group**.
4. Save your changes.

Create a CI class

Create a CI class (table) that is an extension of an existing CI class. Then create identification and reconciliation rules for the new class.

Before you begin

The class that is being extended must have its Is_Extendable field checked, indicating that the class is extendable.

Role required:

- `Itil_admin` and `personalize_dictionary`: Required for editing the dictionary table
- `admin`: Full access

About this task

The CI Class Manager is a centralized location for managing CMDB tables and for creating a class that is derived from another CMDB class. Creating a class requires basic details such as a label and a name. Identification and reconciliation rules are also required to ensure that the class can be successfully identified by the identification engine.

For more information about extending a class and how attributes are derived from a parent class in that process, see [Table extension and classes](#).

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Click **Hierarchy** to expand the CI Classes list.
3. Select the class that the new class is extended from.
4. Click **Add Child Class**.
The **Add Child Class** option appears only if the selected class is extendable.
5. On the **Provide Basic Info** tab, fill out the information and then click **Next**.

Field	Description
Display name	A unique label for the class (such as Laptops or Thin Clients). The label appears on list and form views for the class. Updating the Label field also updates the label record in the language file for the current language. See Field Labels in Data dictionary tables .

Field	Description
	Maximum string length is 80 characters.
Table name	Automatically populated based on the table label and the prefix string 'u_cmdb_ci'. You cannot modify the prefix; however, you can modify the rest of the table name. The name can contain only lowercase, alphanumeric ASCII characters and underscores (_). Maximum string length is 80 characters.
Description	Explanation of the use purpose of the class.
Icon	The icon associated with the class.
Extensible	Indicator of whether this class can be extended.
Principal Class	Denotes whether this class is included in the Principal Class filter . If this class is included in the Principal Class filter, then CIs from this class appear in CI list views when the Principal Class filter is applied.

- On the **Add Attributes** tab, click the + sign and enter details for each new class column.
- Click **Next**.

For description of the different columns in the list view, see [Dictionary entry form](#). To designate an attribute as a CI identifier for class identification, double-click the attribute's Identification Rule column and set it to **true**.

8. On the **Set Identification Rule** tab, examine the **Derived** identification rule and its **Identifier Entries**.
You can click **Replace** to replace the derived rule with a new identification rule and new identifier entries specific to the new class. See [Identification rules](#) for details about identification rules and identifier entries.
9. On the **Dependencies** tab, click **Add dependency** to add [dependent rules](#).
The **Dependencies** tab appears only if there are dependent identification rules for the selected class.
10. On the **Add Reconciliation Rules** tab, click **Add** to create the following rules:
 - a. [Reconciliation Rules](#)
 - b. [Data Refresh Rules](#)
11. On the **Add Suggested Relationships** tab, review the diagram of the class derived suggested relationships.
12. Use the filter to display only inbound, outbound, or specific relationship types.
13. To add a suggested relationship for the class:
 - a. Click **New**.
 - b. In the Add Suggested Relationship dialog box, select a **Relationship** and a **Target Class** for the relationship. **This Class** and the **Target Class** become parent or child in the suggested relationship, based on your selection of the **Relationship**.
 - c. Click **Save**.
When building relationships for the class in the Query Builder, the list of suggested relationships is updated.
14. Click **Done**.

Related tasks

- [Reclassify a CI](#)
- [Delete CIs](#)
- [View and edit class definitions and metadata](#)
- [Update class list in the Principal Class filter](#)

Related concepts

- [Dependent CIs management](#)
- [CMDB Identification and Reconciliation \(IRE\)](#)

Related reference

- [CMDB record types](#)
- [Related Lists of CI components](#)

Reclassify a CI

You can upgrade, downgrade, or switch the class of a CI by modifying its **Class** attribute.

Before you begin

Role required: itil or asset (In general, the roles required to update a CI)

About this task

Each class is defined with a unique set of attributes. This set consists of attributes that were derived from the parent class, and additional attributes defined for the class.

When you reclassify a CI, the following occurs.

1. The set of attributes is adjusted to match the set of attributes of the newly assigned class. Attributes are added or removed as needed.

2. If any attributes are unique to the current class and are not defined in the newly reclassified class, they are lost.
3. A new record with the CI's current sys_id is inserted to the table of the new class, with the appropriate set of attributes for the class (the sys_id of the CI is retained).

Depending on the reclassification, the following occurs.

Downgrade

The CI class is updated to a class that is higher in the class hierarchy, and the newly assigned class is a parent of the current class. For example, reclassifying a CI from the cmdb_ci_server class to the cmdb_ci_computer class.

For example, the cmdb_ci_server class has attributes that the cmdb_ci_computer class does not have. During the downgrade, these attributes and their respective values are not included in the new CI record that is inserted into the cmdb_ci_computer class.

Upgrade

The CI class is updated to a class that is lower in the class hierarchy, and the newly assigned class is a derived child of the current class and has additional attributes. For example, reclassifying a CI from the cmdb_ci_computer class to the cmdb_ci_server.

Switch

The newly assigned class is in a different branch in the class hierarchy and has a different set of attributes than the current class. For example, reclassifying a CI from the cmdb_ci_linux_server class to the cmdb_ci_win_server class.

A switch is a combination of a downgrade and an upgrade. For example if the CI is downgraded to the cmdb_ci_server, and then upgraded to the cmdb_ci_win_server class. Therefore, attributes are lost in the same manner as in a downgrade operation.

Note: Avoid the CI class downgrade and CI class switch operations as those can lead to data loss. When automatic CI reclassification is enabled (which is by default), the [identification process](#) can result in some automatic reclassifications which lead to data loss.

For information about CI reclassification by the Identification and Reconciliation Engine (IRE) and related system properties that control the behavior of automatic CI reclassification, see [Configure CI reclassification during IRE processing](#).

Procedure

1. Locate the CI that you want to reclassify and display it in a list view. You can use the application navigator. Or for example, if the CI is a server, then in the navigation search box, type `cmdb_ci_server.list` to display the CI in the **Servers** view.
2. Ensure that the **Class** field is displayed in the list. If you do not see this attribute, personalize the list to add the **Class** field.
3. Double-click the **Class** value for the CI, and select a new class.
4. Click the green check box to confirm your selection.

Related tasks

- [Create a CI class](#)
- [Delete CIs](#)
- [View and edit class definitions and metadata](#)
- [Update class list in the Principal Class filter](#)

Related concepts

- [Dependent CIs management](#)

Related reference

- [CMDB record types](#)

- [Related Lists of CI components](#)

Delete CIs

You can use the CI Class Manager to delete CIs that are no longer needed.

Before you begin

Role required: `itil_admin`

About this task

For information about policy-based, large scale automated CI deletions, see [Working with CMDB Data Manager](#).

Note: You can't delete base system tables. For information about deleting custom tables, see [Deleting custom tables](#).

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Click **Hierarchy** to expand the CI Classes list and then select the class from which you want to delete CI records.
3. In the class navigation bar on the left, click **CI List**.
4. On the CI List form view, select the CIs that you want to delete. Select the check box in the header to select all the CIs that are visible.
5. Click **Actions on selected rows** and then click **Delete**.
6. Click **Delete** in the **Confirmation** dialog box.

Result

After deleting CIs that a dependent CI depends on, the dependent relationship is also deleted. The dependent CI becomes an orphan and is not immediately deleted. In this situation, the system attempts to cascade delete such orphan dependent CIs to prevent the

accumulation of stale data and maintain the health of the CMDB. For information about how the system manages orphan dependent CIs, see [Management of orphan dependent CIs](#).

Related tasks

- [Create a CI class](#)
- [Reclassify a CI](#)
- [View and edit class definitions and metadata](#)
- [Update class list in the Principal Class filter](#)

Related concepts

- [Dependent CIs management](#)

Related reference

- [CMDB record types](#)
- [Related Lists of CI components](#)

View and edit class definitions and metadata

Use the CI Class Manager as a central location to explore the CMDB class hierarchy, CI table definitions, and class CIs. View the details of each table such as its label and fields, relationships, and all related metadata definitions.

Before you begin

Role required: none

About this task

The [CI Class Manager](#) shows the entire CMDB class hierarchy in a tree-view format, consolidating class definitions into a central location. It lets you display metadata information for a class, such as reconciliation rules, mandatory and recommended fields, and audit templates. You can also select a specific class to view, to modify, or to extend its definition to

create a derived class. For each class, you can directly access CMDB Health settings, identification and reconciliation rules, orphan scorecard, and certificate template, defined for the class.

For more information about extending a class and how attributes are derived from a parent class in that process, see [Table extension and classes](#).

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Click **Hierarchy** to expand the CI Classes list and then select a class to display details for.
3. On the class navigation bar, expand the following items to display further details for the class.

- **Class Info:**

- **Basic Info:** Displays details for the selected class, such as the display and table name, description, and class icon. Lets you edit some of the class definitions, and prevents editing of some details such as the table name.

Role required: `itil` for reading, and `itil_admin` and `personalize_dictionary` for writing.

- **Attributes:** Displays table attributes (columns). Lets you edit those attributes and add new ones. For description of the different attributes in the list view, see [Dictionary entry form](#).

Role required: `personalize_dictionary` for reading and writing

To add an attribute:

- a. Click the **Added** tab and scroll to the bottom of the list.
- b. Double-click **Insert new column**, and enter details for each new class attribute. Set **Identification** to true to designate an attribute as a CI identifier for class identification.
- c. Click **Save**, and fix any errors that appear.

- **Identification and Reconciliation:** Displays and lets you edit, create, and delete identification and [inclusion rules](#), reconciliation and [data refresh rules](#) for the class.

See [CMDB Identification and Reconciliation \(IRE\)](#) for more information.

Role required: itil for reading, and itil_admin (on top of itil) for writing.

- **Dependent Relationships:** Displays and lets you edit, create, and delete hosting and containment relationships for the class. See [CMDB dependent relationship rules](#) for more information.

Role required: itil for reading and itil_admin (on top of itil) for writing.

- **Suggested Relationships:** Displays a diagram of all suggested relationships for the class, and lets you delete or add suggested relationships for the class. Use the navigation tools to increase or decrease the diagram, and to move the diagram on the page. Use the filter to display specific relationship types. See [Suggested class relationships](#) for more information.

Role required: itil.

- **All Relationship Rules:** Displays a combined diagram of all suggested relationships and all dependent relationships for the class. Use the navigation tools to zoom in or out, and to move or center the diagram on the page. Use the filter to display specific relationship categories.
- **Health:** Lets you review and configure CMDB Health-related system properties, scorecards, and rules and settings for all CMDB health KPI and metrics, at the class level. See [CMDB Health](#) for information about enabling and configuring CMDB Health, and displaying health reports.

Role required: Itil for reading and itil_admin (on top of itil) for writing.

- **CI List:** Displays the CIs of the selected class. Lets you create CIs of the selected class and perform other operations such as delete.

Role required: Itil for reading. Writing requirements follow the selected table settings.

Related tasks

- [Create a CI class](#)
- [Reclassify a CI](#)
- [Delete CIs](#)
- [Update class list in the Principal Class filter](#)
- [Create or modify map icons](#)

Related concepts

- [Dependent CIs management](#)

Related reference

- [CMDB record types](#)
- [Related Lists of CI components](#)

Update class list in the Principal Class filter

A Principal Class is a designation for those CMDB classes that are most important or critical for foundational data health and governance in the organization. Manage the list of classes in the Principal Class filter so that those classes are prioritized for tracking, health, certification, lifecycle management, and class list views.

Before you begin

Role required: sn_cmdb_admin or itil_admin, and personalize_dictionary

About this task

Focusing on Principal Classes can improve the CMDB data quality and reliability when you prioritize the health of CIs of critical classes. You can, for example, add to the Principal Class filter essential hardware and software classes such as Linux servers, routers, Oracle, and application services. In a base system, the Principal Class filter doesn't contain any classes. For general guidelines about choosing the classes to be set as Principal Class, see [CMDB schema model](#).

The Principal Class filter has uses across the CMDB, such as:

- In CI lists and dashboards: The Principal Class filter restricts CIs to only those in the filter, so you can focus on those CIs that require attention. For more information about list view filters, see [Save and use filters in a list view](#).
- CMDB Workspace: You can apply the Principal Class filter in the Discovery sources card in the CMDB 360 dashboard. For more information, see [CMDB 360 experience in CMDB Workspace](#).
- Principal CI class agent in Now Assist for CMDB: This agent suggests classes to be set as principal classes, and then automates setup for consistency across environments. For more information, see [Getting advice from Now Assist on CMDB governance](#).

The principal class setting applies only to the current class and is not derived by child classes.

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Select **Hierarchy** to expand the CI Classes list and then select a class to add or remove from the Principal Class filter.
3. On the class navigation bar, navigate to **Class Info > Basic Info**.
4. On the Basic Info form, select or clear **Principal Class**.
5. Select **Save**.

Result

The Principal Class filter is updated with the addition or the removal of the class from the list of classes in the filter. When you apply the Principal Class filter to a Configuration Items list view, only CIs from classes included in the filter, appear.

What to do next

In both of the following scenarios, the list of CIs refreshes to show only CIs whose class is included in the Principal Class filter.

- Scenario 1:
 - 1: In the **Filter navigator**, type `cmdb_ci.list` and then press the Enter key.
 - 2: In the Configuration Items list view, select the **List controls** menu icon, select **Filters** and then select **Principal Class**.
- Scenario 2:
 - 1: Open a Change Request form.
 - 2: Scroll down and select the **Affected CIs** tab. Select **Add**.
 - 3: In the **Add Affected CIs** form, select the **List controls** menu icon, select **Filters** and then select **Principal Class**.

For more information about adding affected CIs to change requests, see [Associated CIs on a change request](#).

Related tasks

- [Create a CI class](#)
- [Reclassify a CI](#)
- [Delete CIs](#)
- [View and edit class definitions and metadata](#)

Related concepts

- [Dependent CIs management](#)

Related reference

- [CMDB record types](#)
- [Related Lists of CI components](#)